Project Part 1: Smart Contract Development

Presentation Deadline: 18 December in class
Individual Viva: Understanding of the Smart Contract and Solidity.
Project Title: Decentralized Freelance Marketplace with Escrow

Description: You are tasked with building the backend Smart Contract for a decentralized freelancing platform where clients can hire freelancers without a middleman, using an **Escrow system** to ensure trust.

System Roles:

1. **Arbiter (Admin):** The platform owner who resolves disputes.
2. **Client:** Users who post jobs and hire freelancers.
3. **Freelancer:** Users who bid on jobs and submit work.

Functional Requirements:

Freelance and client issue

1. User Registration & Reputation System

- All users (Arbiter, Clients, Freelancers) must register.
- Store details: `Name`, `Role`, `Wallet Address`.
- **Constraint:** Freelancers have a **Reputation Score** (starts at 100).
- **Constraint:** A Freelancer cannot register if their wallet address is already associated with a Client account.

2. Post a Job

- Only **Clients** can post jobs.
- Data: `Job Title`, `Category` (e.g., "Dev", "Design"), `Max Budget (in Ether)`, `Deadline (Timestamp)`.
- Each job must have a **Unique Job ID**.
- Initial Status: `Open`.

3. Bidding System

proposed time in place bid

- **Freelancers** can place a bid on `Open` jobs.
- Bid Data: `Bid Amount (Ether)`, `Proposed Time`.
- **Constraint:** Bid Amount cannot exceed the Client's `Max Budget`.
- **Constraint:** Freelancers cannot bid if their **Reputation Score is below 50**.

4. Hire & Escrow (Payable Feature)

0 is job open and
1 means in progress

- The **Client** selects a bid to hire a Freelancer.

- **Challenge (Escrow):** Upon hiring, the Client **must send the exact Bid Amount** in Ether to the Smart Contract. The contract holds this money (Balance matches the job value).
- Status updates to: `InProgress`.
- If the Ether sent does not match the bid amount, the transaction must **revert**.

5. Work Submission & Completion

- **Freelancer** marks the job as `Completed`.
- **Client** approves the work.
- **Constraint (Dynamic Fee & Transfer):**
    - Once approved, the Smart Contract transfers the funds to the Freelancer.
    - **Platform Fee Deduction:**
        - If Job Value < 1 Ether: Deduct **2%** fee **approximately** (remains in contract for Arbiter).
        - If Job Value >= 1 Ether: Deduct **1%** fee **approximately**.
    - Update Freelancer Reputation: **+10 points**.
    - Status updates to: `Closed`.

6. Dispute & Resolution (Arbiter Only)

- A Client can mark a job as `Disputed` instead of approving it.
- **Arbiter** reviews the dispute and decides the outcome:
    - **Refund Client:** 100% of funds return to Client. Freelancer Reputation: **-20 points**.
    - **Pay Freelancer:** Funds transfer to Freelancer (minus platform fee).
- Status updates to: `Resolved`.

Project Part 2: Decentralized Application (DApp) Interface

Presentation Deadline: **TBA**
Individual Viva: Understanding of the Project, DApp, and Ethereum blockchain.

Description: Now that your Smart Contract is ready, you must build the client-side interface using **HTML/CSS/JavaScript (or a framework like React/Vue)**, communicating via **Web3.js/Ethers.js** and **MetaMask**.

Requirements:

- *Note:* Some features are open-ended, so think logically about real-life scenarios, what makes sense, and complete the Project accordingly with your own restrictions if needed.

1. Role-Based Dashboard

- The UI must detect the active MetaMask account and display the correct view (Client, Freelancer, or Arbiter).
- **Registration:** A form for new users to register.
- **Validation:** Prevent submitting if fields are empty.

2. Job Marketplace (Filtering & Sorting)

- Display a list of all `Open` jobs.
- **Sorting:** Sort jobs by **Highest Budget** first.
- **Filtering:** Add a dropdown to filter jobs by **Category** (e.g., Show only "Design" jobs).
- *Note:* Perform this sorting/filtering in an **efficient manner**.

3. Client Interaction (Posting & Hiring)

- **Post Job Form:** Inputs for Title, Category, Budget, and Date. (Convert Date to Unix Timestamp for Solidity).
- **View Bids:** When a Client clicks a specific job, show a list of received bids.
- **"Hire" Action:** A "Hire" button next to each bid.
    - **Payment Trigger:** Clicking "Hire" must trigger a MetaMask transaction asking the user to pay the **Bid Amount**.
    - *Success:* Show a distinct "Funds in Escrow" badge after the transaction confirms.

4. Freelancer Interaction (Bidding)

- **Bid Form:** Only visible to Freelancers.
- **Validation:** Prevent the "Place Bid" button from working if the input amount is higher than the Job's budget.

- **My Jobs:** A section showing jobs where the Freelancer is hired. Include a "Submit Work" button.

5. Arbiter Panel

- A private section visible *only* to the Arbiter.
- Shows a list of jobs with status `Disputed`.
- Two buttons: **"Refund Client"** and **"Pay Freelancer"**.
- Display the **Total Platform Fees** collected by the contract (Contract Balance).

Technical Constraints:

- **No Database:** Do not use MongoDB or SQL. All job details, bids, and user profiles must be fetched directly from the Blockchain.
- **Auto-Refresh:** The UI should update automatically (e.g., using Events) when a bid is placed or a status changes, without requiring a full page reload.
- **Tools:** Use **Ganache** for the local blockchain and **Truffle** for deployment.